DESIGN DOCUMENT

Development of an Ethernet-Based Vehicle Monitoring System Using Linux Socket Programming

1. Introduction

Automotive electronic control units (ECUs) exchange information to support vehicle monitoring and control. Communication between a vehicle ECU and a dashboard is an important example of data exchange between automotive applications.

This project implements a simulated Vehicle ECU and Dashboard ECU using Linux socket programming in the C programming language. The Vehicle ECU generates sample vehicle parameters and transmits them to the Dashboard ECU using TCP or UDP.

Wireshark is used to capture and analyze the network traffic.

2. Objectives

The objectives of this project are:

- To understand Linux socket programming using C.
- To implement TCP client-server communication.
- To implement UDP sender-receiver communication.
- To simulate vehicle speed, engine RPM, and steering angle.
- To display received vehicle parameters on a Dashboard ECU.
- To capture and analyze TCP and UDP traffic using Wireshark.
- To compare the behavior of TCP and UDP.
- To test communication under different operating conditions.

3. System Requirements

3.1 Software Requirements

Component	Description
Operating System	Ubuntu Linux
Programming Language	C
Compiler	GCC
Network API	Linux/POSIX socket API
Packet Analyzer	Wireshark
Build Tool	GNU Make
Network Interface	Loopback interface (`lo`)

3.2 Network Configuration

Parameter	Configuration
IP Address	127.0.0.1
Server Port	5000
TCP Socket Type	SOCK_STREAM
UDP Socket Type	SOCK_DGRAM

The system is configured to run both applications on the same computer using the loopback interface.

4. System Architecture

The system contains two main software components:

Vehicle ECU

The Vehicle ECU generates simulated vehicle parameters, including: - Vehicle speed in kilometres per hour. - Engine speed in revolutions per minute. - Steering angle in degrees.

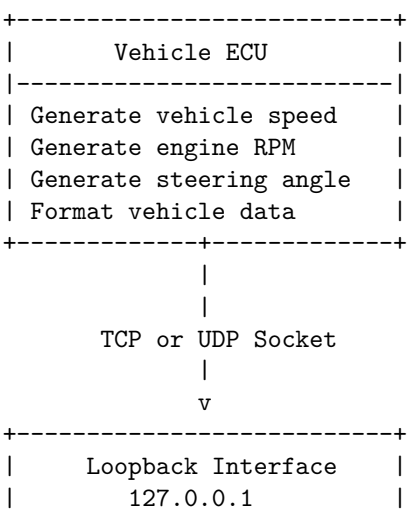
It formats these values into a text message and transmits the message to the Dashboard ECU.

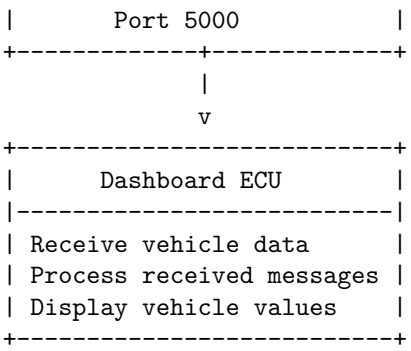
Dashboard ECU

The Dashboard ECU receives the transmitted message and displays the vehicle parameters in the terminal.

The Dashboard ECU acts as a TCP server when TCP mode is used and as a UDP receiver when UDP mode is used.

5. System Block Diagram





6. TCP Communication Design

TCP is a connection-oriented transport protocol.

In this implementation:

1. The Dashboard ECU creates a TCP socket.
2. It binds the socket to IP address `127.0.0.1` and port `5000`.
3. It calls `listen()` to wait for client connections.
4. It calls `accept()` to accept a connection from the Vehicle ECU.
5. The Vehicle ECU creates a TCP socket and calls `connect()`.
6. Vehicle data is transmitted using `send()`.
7. The Dashboard ECU receives data using `recv()`.
8. The connection is closed when the client disconnects.

TCP provides reliable, ordered byte-stream delivery. Application messages are separated using newline characters in this project.

7. UDP Communication Design

UDP is a connectionless transport protocol.

In this implementation:

1. The Dashboard ECU creates a UDP socket.
2. It binds the socket to `127.0.0.1:5000`.
3. The Vehicle ECU creates a UDP socket.
4. It transmits vehicle data using `sendto()`.
5. The Dashboard ECU receives datagrams using `recvfrom()`.
6. The received vehicle values are displayed in the terminal.

UDP does not establish a connection before transmitting data and does not guarantee delivery or ordering.

8. Vehicle Data Format

The Vehicle ECU generates simulated parameters and formats them as a text message.

Example:

```
Speed=56 km/h, RPM=4259, Steering=2.0 degrees
```

The values are generated programmatically and do not represent measurements from an actual vehicle.

The Vehicle ECU transmits the message approximately once per second.

9. Software Modules

9.1 Vehicle ECU Module Responsibilities: - Create the communication socket. - Configure the destination address. - Establish a TCP connection when using TCP. - Generate simulated vehicle parameters. - Format and transmit vehicle data. - Report transmission errors.

9.2 Dashboard ECU Module Responsibilities: - Create the receiving socket. - Bind the socket to the configured address and port. - Accept TCP connections when using TCP. - Receive incoming messages. - Separate and display vehicle data. - Handle client disconnections and receiving errors.

9.3 Packet Analysis Module Wireshark is used to: - Capture packets on the loopback interface. - Filter TCP packets using `tcp.port == 5000`. - Filter UDP packets using `udp.port == 5000`. - Inspect IP addresses and transport-layer ports. - Examine application payloads. - Observe TCP acknowledgments and connection behavior.

10. Build and Execution

Compile the programs from the source directory:

```
cd ~/ethernet-vehicle-monitoring/src
make
```

Run the TCP Dashboard ECU:

```
./dashboard_ecu_tcp
```

In another terminal, run the TCP Vehicle ECU:

```
./vehicle_ecu_tcp
```

For UDP, run the original Dashboard and Vehicle ECU executables:

```
./dashboard_ecu
```

```
./vehicle_ecu
```

Run only one Dashboard server at a time because both implementations use port `5000`.

11. Testing Strategy

The system can be evaluated using the following scenarios:

1. Normal TCP communication.
2. Normal UDP communication.
3. Dashboard ECU unavailable.
4. Invalid or unreachable destination address.
5. Continuous vehicle-data transmission.
6. TCP and UDP packet analysis using Wireshark.

The observed outcome of each test should be recorded in the Validation Report.

12. Limitations

- Vehicle parameters are simulated.
- Both applications run on the same computer.
- Communication uses the loopback interface rather than a physical Ethernet network.
- The project does not connect to an actual automotive ECU or vehicle bus.
- The terminal provides a basic dashboard rather than a graphical vehicle interface.

13. Future Scope

Possible extensions include:

- Communication between two computers over a physical Ethernet network.
- Integration with an STM32-based Vehicle ECU.
- A graphical dashboard displaying vehicle parameters.
- Logging vehicle data to a file or database.
- Adding message timestamps and sequence numbers.
- Implementing connection-loss detection and reconnection.
- Integrating CAN bus data with the monitoring application.

14. Conclusion

This project demonstrates TCP and UDP socket programming in Linux using C. A simulated Vehicle ECU generates speed, RPM, and steering-angle values and transmits them to a Dashboard ECU.

Wireshark is used to examine the transmitted network traffic and compare the two transport protocols.

The implementation provides a foundation for understanding application-level vehicle monitoring communication. The current results apply to local loopback communication and simulated vehicle parameters.